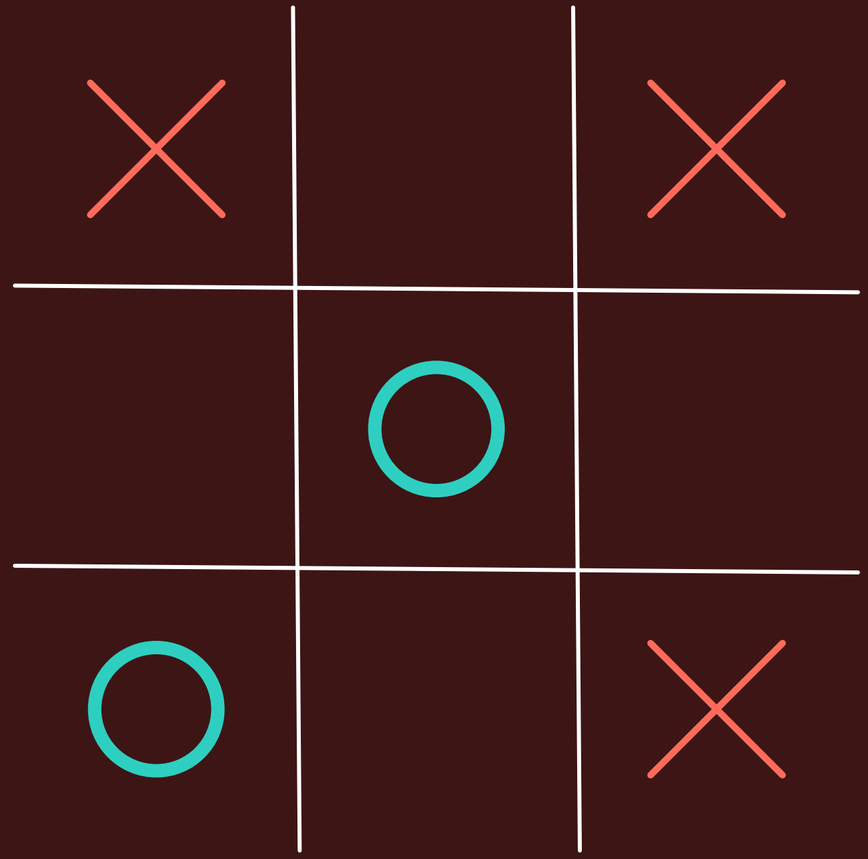
REVIEW - 1

Tic-Tac-Toe Game Using Alpha-Beta Pruning



STUDENTS	Joy Peswani	25WU0102110
	Harshit Jain	25WU0102094
	Devyani pawar	25WU0102069
	K.Prajvesh	25WU0102114
	Abhigna Sai	25WU0102075
BRANCH	AIML Rhinos	
SUBJECT	Foundation in Artificial Intelligence	

INTRODUCTION

What Is This Project About?



Tic-Tac-Toe

A two-player game on a 3×3 grid. Players alternate placing X and O, each aiming to line up three marks in a row, column, or diagonal.



Minimax

A recursive search algorithm where the AI assumes the opponent always plays optimally — maximizing its own outcome while minimizing the opponent’s.



Alpha-Beta Pruning

An optimization of Minimax that cuts off branches which cannot influence the final decision, sharply reducing computation without sacrificing accuracy.

PROBLEM STATEMENT

The Challenge

Intelligent Decisions

Build an AI opponent that selects optimal moves, not random ones.

Future Move Prediction

The AI must evaluate possible future game states before committing to a move.

Avoid Redundant Work

Minimize unnecessary game-state evaluations to improve efficiency.

Competitive Experience

Provide a challenging, engaging Human-vs-AI gameplay experience.

OBJECTIVES

01

Develop Human vs. AI Play

Build a fully playable game interface.

02

Implement Minimax

Recursive adversarial search for optimal moves.

03

Add Alpha-Beta Pruning

Reduce search space without losing accuracy.

04

Select Optimal Moves

AI always chooses the best available move.

05

Reduce Unnecessary Search

Prune branches that cannot influence the outcome.

06

Analyze Performance

Compare nodes explored and execution time.

RESEARCH GAP

What’s Missing in Existing Work?

01

Limited Performance Analysis

Many implementations demonstrate the algorithms without detailed node-count or timing comparisons.

02

Scarce Educational Comparisons

Side-by-side Minimax vs. Alpha-Beta analysis is rarely presented for student audiences.

03

Missing Decision Visualization

AI decision-making and pruned branches are seldom visualized for learners.

LITERATURE REVIEW

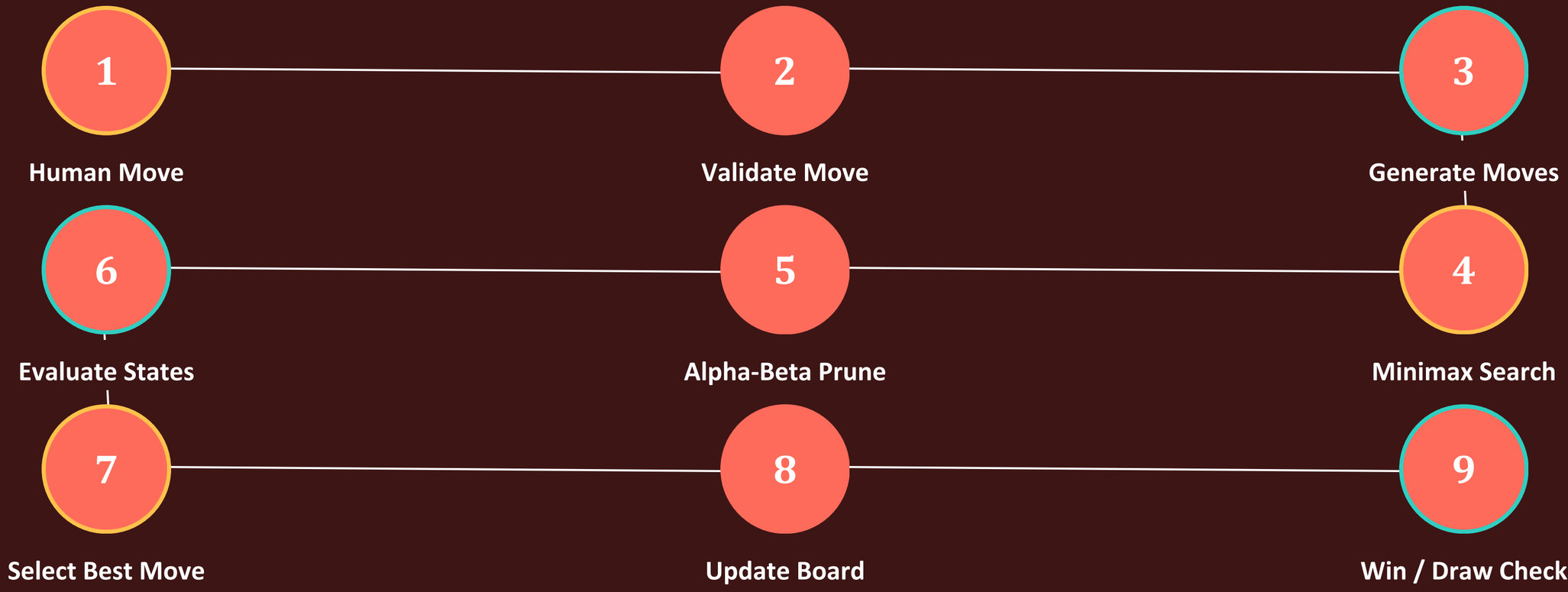
Prior Research

Author / Year	Approach	Algorithm	Key Finding
Shannon (1950)	Computer chess programming	**Minimax**	First formal proposal of minimax for game-playing AI
McCarthy (1956)	Adversarial search foundations	**Alpha-Beta Pruning**	Introduced pruning to reduce the minimax search space
Knuth & Moore (1975)	Analysis of game tree search	**Alpha-Beta**	Proved alpha-beta examines O(b^(d/2)) nodes vs. O(b^d)
Russell & Norvig (2020)	Artificial Intelligence: A Modern Approach (4th Ed.)	**Minimax + A-B**	Standard reference for adversarial search in AI curricula

PROPOSED SYSTEM

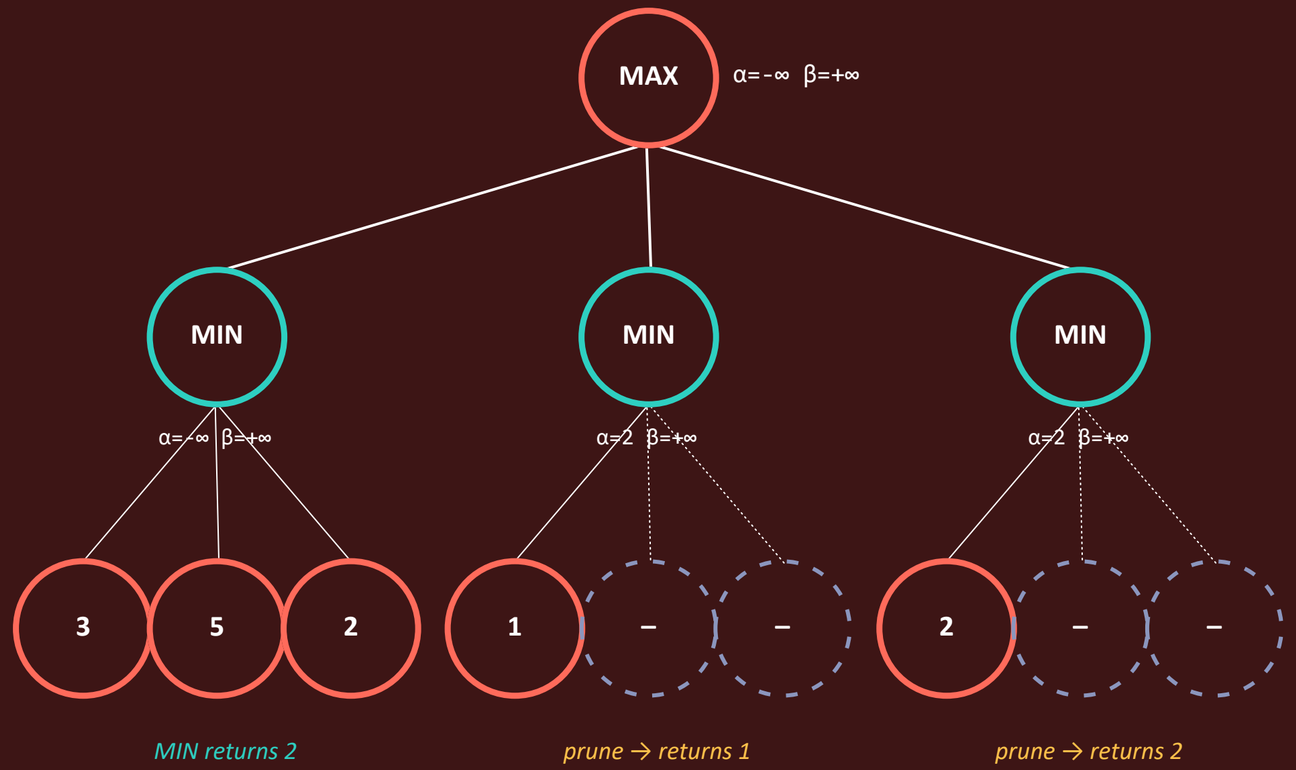
System Workflow

The AI pipeline runs a clear sequence from human input to board update — validating moves, searching game states with Minimax and Alpha-Beta Pruning, and checking for a win or draw before the next turn.



ALGORITHM

Alpha-Beta Minimax



Root evaluates max(2, 1, 2) = 2

Evaluated node

Pruned (not evaluated)

How It Works

Maximizing player (AI)

Tries to maximize its own score at each MAX node.

Minimizing player (opponent)

Tries to minimize the AI's score at each MIN node.

Alpha (α)

The best value found so far for the maximizer.

Beta (β)

The best value found so far for the minimizer.

Pruning condition

When α ≥ β, the remaining branches are skipped — they cannot change the final decision.

IMPLEMENTATION

Core Implementation in Python

```
def alphabeta(board, depth, alpha, beta, maximizing):
   if check_win(board): return score(board)
   if depth == 0: return evaluate(board)

if maximizing:
   max_eval = -float('inf')
   for move in get_moves(board):
      eval = alphabeta(move, depth-1, alpha, beta, False)
      max_eval = max(max_eval, eval)
      alpha = max(alpha, eval)
      if beta <= alpha: break   # prune
   return max_eval
else:
   min_eval = float('inf')
   for move in get_moves(board):
      eval = alphabeta(move, depth-1, alpha, beta, True)
      min_eval = min(min_eval, eval)
      beta = min(beta, eval)
      if beta <= alpha: break   # prune
   return min_eval
```

Key Components

- **Python**
  Core implementation language.
- **Game board**
  3×3 list-based board representation.
- **Minimax function**
  Recursive adversarial search.
- **Alpha-Beta pruning**
  Branch elimination for efficiency.
- **Win / draw detection**
  Terminal game-state checks.
- **User input**
  Handles the human player’s moves.

RESULTS & PERFORMANCE

Performance Comparison

Empirical Node Exploration & Computational Latency from Empty Board

Parameter	Minimax (No Pruning)	Minimax + Alpha-Beta Pruning
Nodes Explored (Empty Board)	549,946	**20,866 (96.2% fewer nodes)**
Execution Time (Empty Board)	~800.0 ms	**~25.0 ms (32x faster)**
Pruned Branches (Cutoffs)	0 (None)	**7,692 subtrees skipped**
Search Efficiency	Baseline (100% search)	**96.2% search space reduction**
Decision Optimality	Score 0 (Guaranteed Draw)	**Score 0 (Guaranteed Draw - 100% Identical)**

KEY EMPIRICAL FINDING (FOR PRESENTATION QUOTING):

Measured during live execution from an empty board. Pure Minimax explores 549,946 nodes to find the optimal move.
Alpha-Beta Pruning evaluates the identical optimal move while exploring only 20,866 nodes and executing 7,692 cutoffs.
This eliminates 529,080 redundant states (96.2% search reduction) with ZERO loss of accuracy - proving 100% minimax optimality.

SYSTEM PROTOTYPE

Interactive Web Application Prototype

Educational Two-Pane Architecture with Live Algorithm Visualization

Dual-Pane Playable Game Interface

* Interactive 3x3 board with real-time score tracking (Human / AI / Draws).
* Dynamic status indicator: 'Your Turn', 'AI Thinking...', and winning line glow.
* Live Algorithm Selector: Minimax vs. Minimax + Alpha-Beta Pruning.
* Difficulty Engine: Hard (unbeatable full depth), Medium, and Easy modes.
* First-Move Toggle: Supports both Human-First and AI-First openings.

Interactive Decision Tree Visualization

* Live SVG tree rendering AI (MAX) and Human (MIN) lookahead states.
* Evaluated Nodes: Rendered as clean solid circles with utility scores.
* Pruned Branches: Rendered as dashed circles with scissors icon (Cutoff).
* Golden Edge Highlighting: Visually highlights the optimal chosen move path.
* Subtree Focus: Click any candidate move to isolate and explore its subtree.

Boundary-Aware Hover Tooltips

* Zero Visual Clutter: Details hidden until mouse hover over any node.
* Instant Node Metadata: Node Role (MAX AI / MIN Human), Move Cell, and Score.
* Live Alpha-Beta Bounds: Displays current [Alpha, Beta] bounds at every node.
* Smart Collision Detection: Tooltip automatically flips left/up near edges.
* Hard Viewport Clamping: Prevents tooltip from ever sliding outside screen.

Real-Time Search Analytics & Controls

* Search Volume Bar Chart: SVG chart comparing nodes explored per turn.
* Candidate Move Map: Mini-board ranking all legal moves (+8, -8, 0).
* Expandable Analytics Pane: Smooth draggable divider with 1-click collapse.
* Edge-to-Edge Fullscreen Modal: Press ESC or 'Back to Normal' to exit.
* Real-Time Efficiency Tracker: Reports cumulative % node savings across game.

VERIFICATION & TESTING

Rigorous Testing & Proof of Optimality

Automated Pytest Suite & Exhaustive Game-Tree Simulation (0 AI Losses)

16/16 Automated Tests in Pytest

* 100% deterministic test pass rate in under 1.9 seconds execution time.
* Comprehensive validation of all 8 winning rows, columns, and diagonals.
* Terminal draw detection when board fills with no three-in-a-row.
* Illegal move rejection: Occupied cell and out-of-bounds protection.
* Flask REST endpoints (/api/move, /api/reset) verified with WSGI test client.

Mathematical Equivalence Proof

* Formal verification: Minimax and Alpha-Beta return identical best scores.
* Tested across 30 randomized game positions with varying turn depths.
* Verified Alpha-Beta explores <= Minimax nodes across 100% of test cases.
* Proves Alpha-Beta pruning is 100% safe: Never prunes an optimal path.
* Decision tree integrity: Node depths, alpha bounds, and beta cutoffs verified.

Exhaustive Simulation: 0 AI Losses

* Human-First Exhaustive Search: Simulated all 569 possible human paths.
* AI-First Exhaustive Search: Simulated all 73 possible opening AI paths.
* Result: 0 AI LOSSES across all complete games played to terminal states.
* Proven Outcome: The AI plays flawlessly and is mathematically unbeatable.
* Human player can at best achieve a draw with perfect counter-play.

Engineering & Deployment Reliability

* Modular Python architecture: game.py, ai.py, and app.py separated cleanly.
* macOS AirPlay Port Conflict Handler: Auto-detects port 5000 and uses 5001.
* Zero external frontend libraries: Pure vanilla HTML, CSS, and modern JS.
* 1-Click macOS Double-Click Launcher: start.command runs venv & server.
* Client resilience: Recovers state instantly if connection is interrupted.

CONCLUSION

Project Achievements & Future Scope

Foundations of Artificial Intelligence - Review 1 Deliverables

Core Findings

> 96.2% Node Reduction

Alpha-Beta Pruning slashes search space from 549,946 to 20,866 nodes from the initial empty board.

> 32x Computational Speedup

Search duration drops from ~800 ms down to ~25 ms, enabling instantaneous real-time responses.

> Perfect Decision Equivalence

Both algorithms output identical optimal moves. Pruning introduces zero heuristic compromise.

> Mathematically Unbeatable AI

Exhaustive testing confirms 0 losses across all possible human opponent move permutations.

Educational Value

> Interactive Adversarial Tree

Visualizes abstract game theory with live MAX and MIN nodes and dashed pruning cutoffs.

> Intuitive Scoring Mechanics

Explains utility scores (+10 - depth for faster wins, depth - 10 to prolong losses, 0 for draws).

> Clear Cutoff Intuition

Demonstrates why branches are pruned immediately when Beta <= Alpha without losing accuracy.

> Dual Algorithm Comparison

Live side-by-side metrics table allows students to see the exact power of optimization in real time.

Future Scope

> Higher Dimension Grids

Extend the engine to 4x4 and 5x5 Tic-Tac-Toe boards using depth-limited iterative deepening.

> Heuristic Evaluation

Implement non-terminal heuristic evaluation functions for deeper search without exponential blowup.

> Transposition Tables

Incorporate Zobrist hashing to cache already-evaluated game positions and avoid repeated search.

> Move Ordering Optimization

Sort candidate moves dynamically to trigger earlier Alpha-Beta cutoffs and further reduce node counts.